

Project: Prototype 2 - Advanced Architectural Structures

Team 1e

Team Members

- Nicolas Felipe Arciniegas Lizarazo
- Karen Lorena Guzman Del Rio
- Juan David Chacon Muñoz
- Adrian Yebid Rincon
- Pablo Felipe Sandoval Menjura
- Julio Cesar Albadan Sarmiento

Software System

Name

FitBeat

Logo



Description

FitBeat is a distributed fitness platform that synchronizes workout sessions with Spotify playback. A user can sign up or log in, connect a Spotify account, configure music preferences, and start a training session. During the session, playback can be controlled in real time, while business events are emitted and consumed asynchronously for achievements, analytics, and notifications.

The current prototype uses a microservice architecture with dedicated persistence per domain, an API Gateway for entry-point orchestration, and a message broker for asynchronous workflows.

Prototype 2 Requirements Coverage

Functional completeness

The team-defined functional scope currently includes:

- User registration and authentication.
- Spotify OAuth connection.
- Training session creation and playback control.
- Achievement evaluation and retrieval.
- Event-driven notification processing.

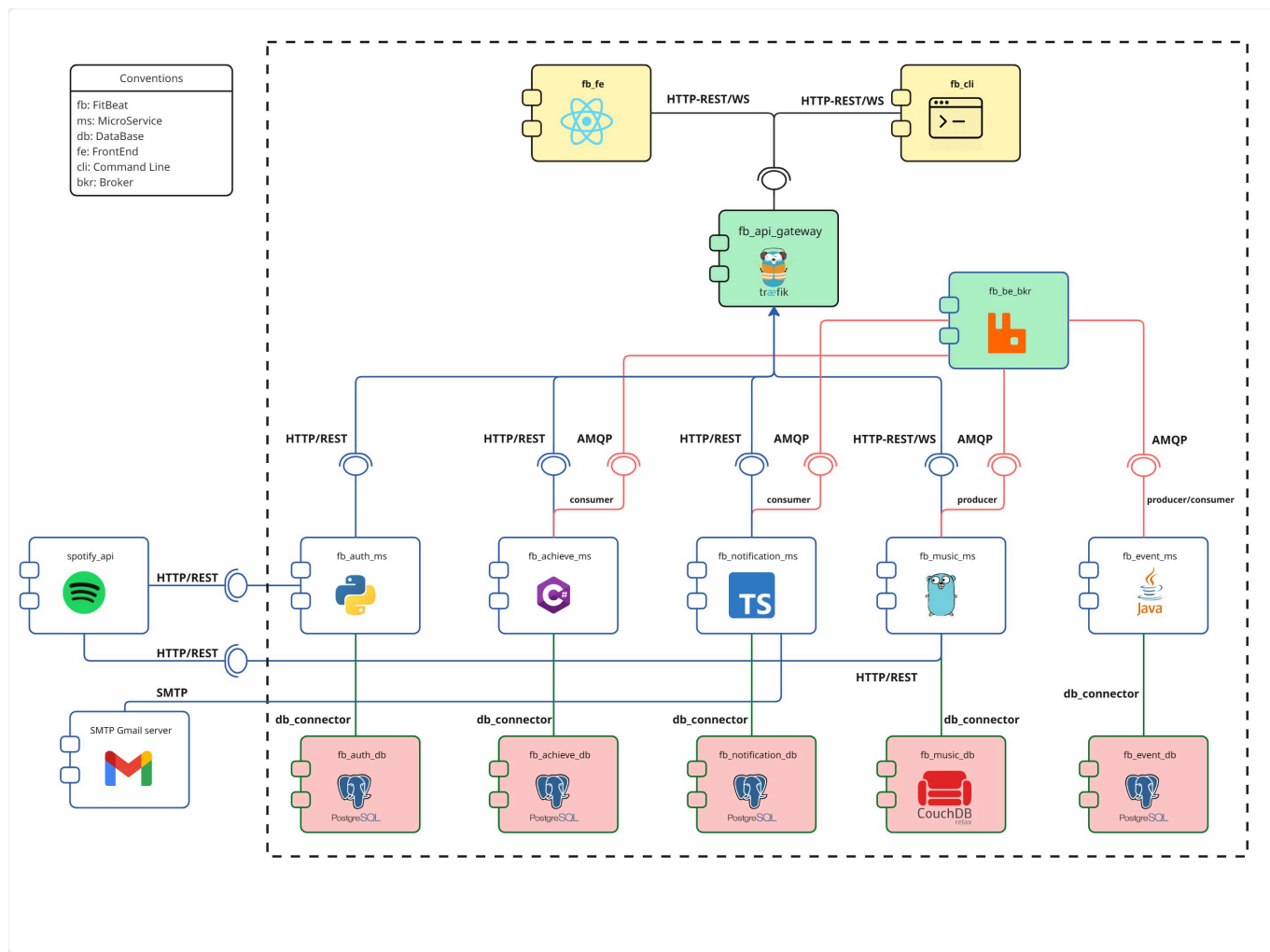
Non-functional requirements mapping

- Distributed architecture: satisfied through independently deployable services.
- At least two presentation components: satisfied (`frontend/web` and `frontend/cli`).
- Web frontend with SSR subarchitecture: partially satisfied; current web frontend is React + Vite (CSR-oriented), SSR migration is pending if strict SSR validation is required.
- At least five logic components: satisfied (`user-service`, `music-service`, `achievements-service`, `notification-service`, `event-processor`).
- At least one communication/orchestration component: satisfied (`traefik` API Gateway, plus `rabbitmq` for async orchestration).
- At least four data components including relational and NoSQL: satisfied (multiple PostgreSQL databases + CouchDB).
- At least one asynchronous processing component: satisfied (`event-processor`, plus async consumers in achievements/notifications).
- HTTP-based connectors: satisfied (REST APIs through gateway and direct service endpoints).
- At least five general-purpose programming languages: satisfied (Python, Go, C#, TypeScript/JavaScript, Java).
- Container-oriented deployment: satisfied (Docker Compose deployment model).

Architectural Structures

1. Component-and-Connector Structure

C&C View



Architectural styles used

- Layered architecture:
 - The system can be organized into four tiers: presentation, communication/orchestration, logic, and data.
 - Each tier has a clear responsibility boundary and depends on explicit contracts.
- Client-server style:
 - Frontends act as clients and backend services expose server endpoints.
 - Requests are served mainly through HTTP APIs, with WebSocket support for real-time interaction.
- Microservice-based architecture:
 - Business capabilities are split into independently deployable services.
 - Each service owns its domain logic and can evolve with minimal coupling to other services.
- Distributed architecture:
 - Components run in separate containers and communicate through network connectors.
 - The deployment can be distributed across nodes while preserving service contracts.

Architectural patterns used

- 4-tier architecture pattern:
 - Presentation tier: `frontend/web`, `frontend/cli`.
 - Communication/orchestration tier: `traefik`, `rabbitmq`.
 - Logic tier: domain microservices.
 - Data tier: PostgreSQL databases and CouchDB.

- API Gateway pattern:
 - `traefik` centralizes ingress, hides internal topology, and routes requests by URL path prefix.
 - This reduces frontend coupling to internal service locations.
- Broker pattern:
 - `rabbitmq` decouples event producers and consumers.
 - Asynchronous processing avoids blocking user flows and improves resilience for background workloads.
- Database-per-service pattern:
 - Every core service has dedicated persistence boundaries.
 - This prevents direct shared-schema coupling and supports domain autonomy.
- Repository pattern (where applicable):
 - Services encapsulate persistence access behind service-internal data-access modules.
 - This keeps business logic separated from storage-specific operations.

Architectural elements and relations

Presentation components

- `frontend/web` (React + Vite):
 - Main user-facing web interface.
 - Communicates with backend services through the gateway.
- `frontend/cli` (Node.js):
 - Command-line client for API interactions.
 - Uses the same gateway entry point in container network contexts.

Communication and orchestration components

- `traefik` (`fb_gateway`):
 - Central entry point for HTTP and WebSocket traffic.
 - Routes by path prefix:
 - `/api/auth, /auth, /users` -> `component_a` (`user-service`)
 - `/api/v1` -> `music_service`
 - `/achievements` -> `achievements_service`
 - `/notifications` -> `notification_service`
- `rabbitmq` (`fb_rabbitmq`):
 - Event broker for asynchronous interactions.
 - Decouples event producers and consumers.

Logic-type components

- `user-service` (FastAPI, Python):
 - Authentication, local user management, JWT, Spotify OAuth token management.
 - Relational persistence in PostgreSQL (`fb_users_db`).
- `music-service` (Go):
 - Training session and playback orchestration with Spotify integration.
 - Document persistence in CouchDB (`fb_music_db`).
 - Publishes domain events to RabbitMQ.

- **achievements-service** (.NET 8):
 - Achievement rules and badge assignment.
 - Relational persistence in PostgreSQL (**fb_achievements_db**).
 - Consumes asynchronous events for gamification updates.
- **notification-service** (TypeScript/Node.js):
 - Notification generation and persistence.
 - Email dispatch and user-notification query endpoints.
 - Relational persistence in PostgreSQL (**fb_notification_db**).
 - Consumes asynchronous events from RabbitMQ.
- **event-processor** (Spring Boot, Java):
 - Asynchronous event processing and metric/stat aggregation.
 - Relational persistence in PostgreSQL (**fb_event_db**).

Data components

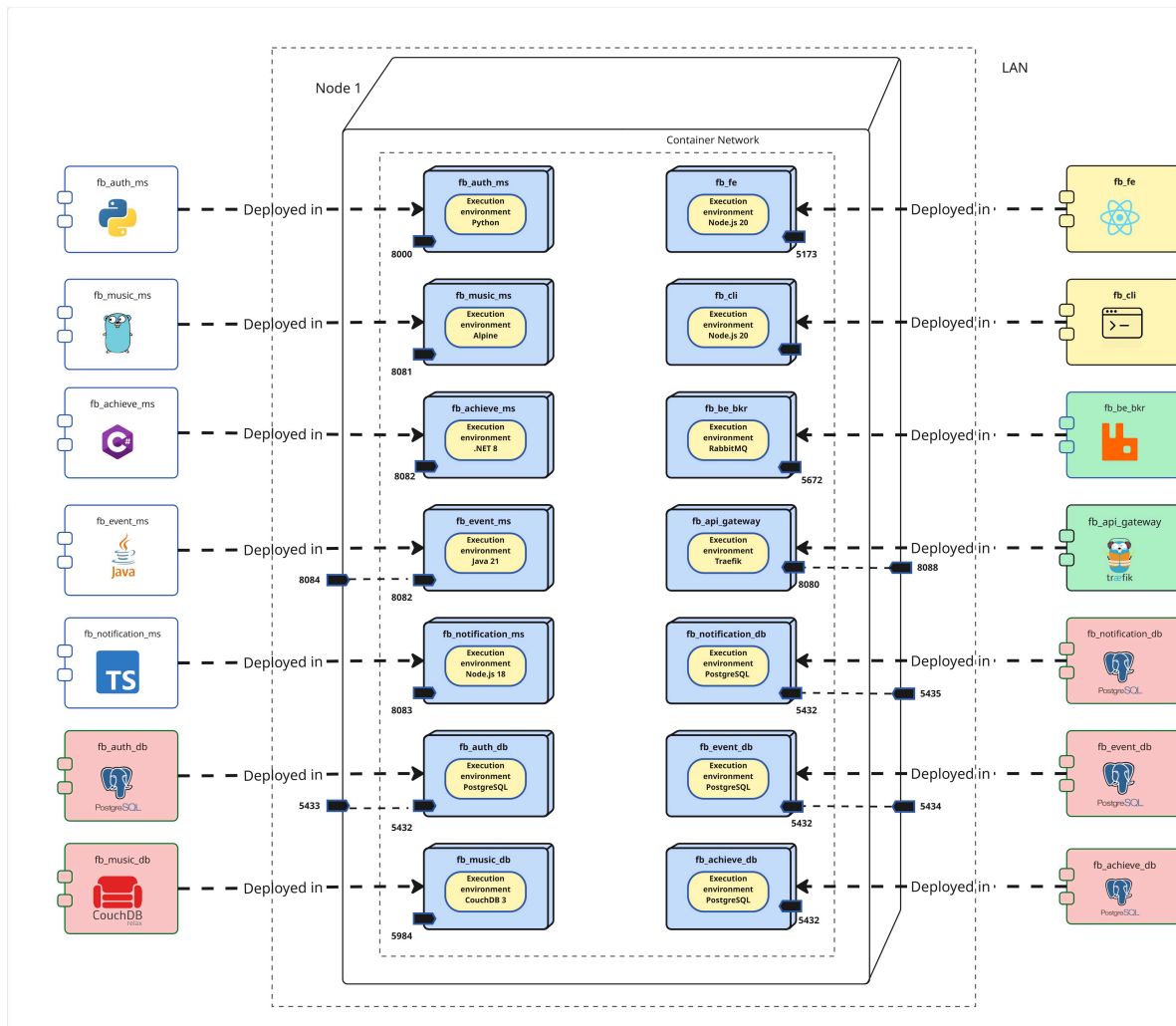
- **fb_users_db** (PostgreSQL): user and auth-related data.
- **fb_music_db** (CouchDB): workout session documents and music-related records.
- **fb_achievements_db** (PostgreSQL): achievements and progression data.
- **fb_notification_db** (PostgreSQL): notification records and delivery outcomes.
- **fb_event_db** (PostgreSQL): processed-event tracking and derived metrics.

External system

- Spotify API:
 - OAuth integration (via user service).
 - Playback/search integration (via music service).

2. Deployment Structure

Deployment View



Architectural elements and relations

- Deployment model is container-oriented on a Docker host.
- Main Docker networks:
 - `component_a_network`: internal service-to-service communication.
 - `gateway_network`: service exposure for gateway-routed traffic.
- Core deployment allocation (`deployed_in`):
 - Gateway logic -> `fb_gateway`.
 - User logic -> `fb_users_ms`.
 - Music logic -> `fb_music_ms`.
 - Achievements logic -> `fb_achievements_ms`.
 - Notification logic -> `fb_notification_ms`.
 - Event processing logic -> `fb_event_processor`.
 - Web presentation -> `fitbeat_frontend`.
 - CLI presentation -> `fitbeat_cli`.
 - Persistence components -> dedicated DB containers.
 - Async transport -> `fb_rabbitmq`.

Runtime environments by component

- `fb_gateway`: Traefik v3.1 runtime.
- `fb_users_ms`: Python 3.11 + Uvicorn ASGI server.

- `fb_music_ms`: Go compiled binary on Alpine runtime.
- `fb_achievements_ms`: .NET 8 ASP.NET Core runtime (`dotnet`).
- `fb_notification_ms`: Node.js 18 runtime (`node dist/index.js`).
- `fb_event_processor`: Java 21 runtime (`java -jar app.jar`).
- `fitbeat_frontend`: Node.js 20 + Vite dev server.
- `fitbeat_cli`: Node.js 20 runtime.

Port exposure summary

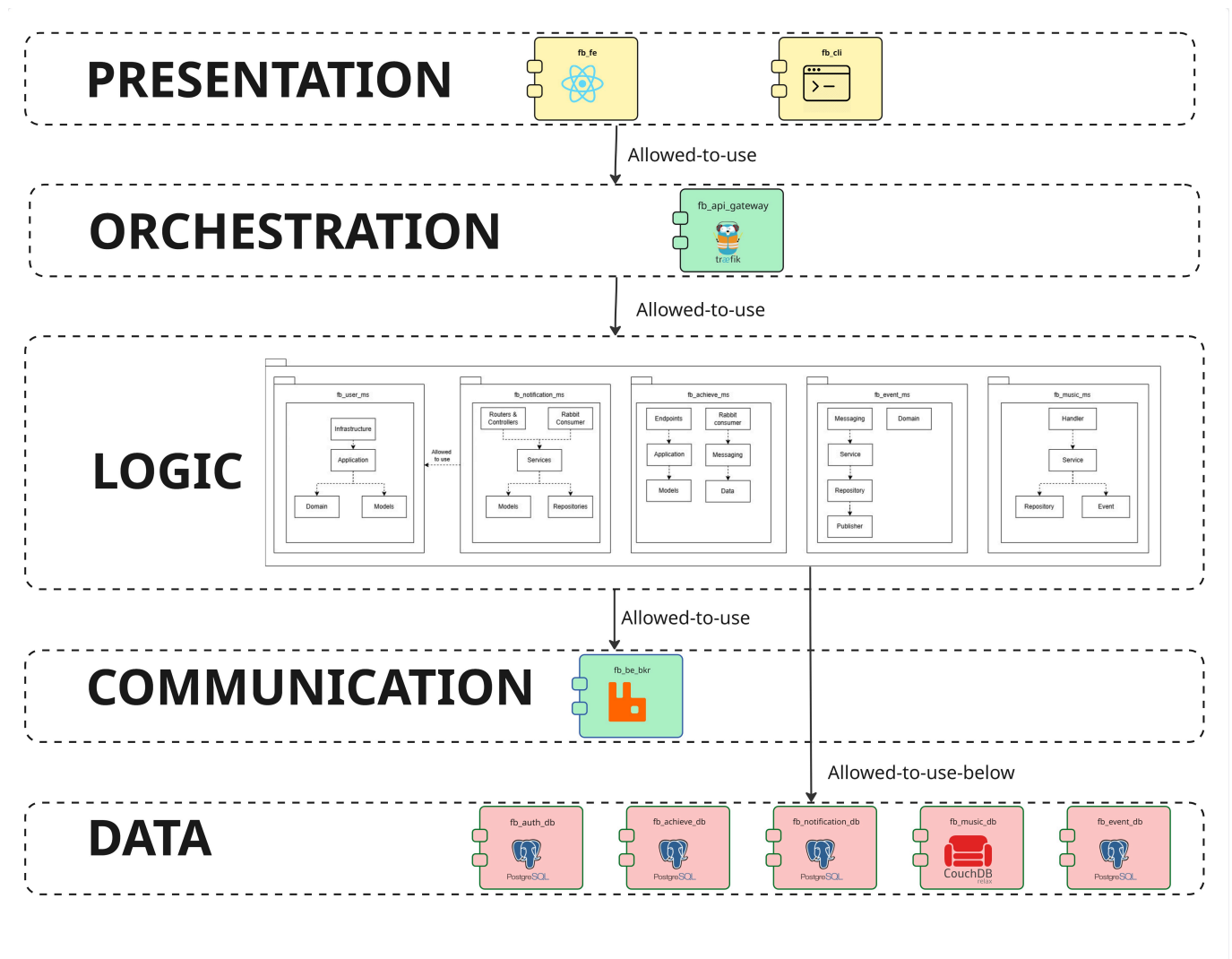
- Gateway: `8090:80`, Dashboard: `8088:8080`.
- User service: `8000:8000`.
- Music service: `8081:8081`.
- Achievements service: `8082:8082`.
- Notification service: `8083:8083`.
- Event processor: `8084:8082`.
- Databases:
 - `fb_achievements_db`: `5432:5432`
 - `fb_users_db`: `5433:5432`
 - `fb_event_db`: `5434:5432`
 - `fb_notification_db`: `5435:5432`
 - `fb_music_db`: `5984:5984`
- RabbitMQ: `5672:5672`, management UI `15672:15672`.

Deployment patterns used

- Container-oriented deployment:
 - Every major component runs in an isolated container with explicit runtime dependencies.
 - This enables reproducible local deployment and portable execution environments.
- Reverse proxy + path-based routing:
 - `traefik` acts as ingress and forwards requests based on path rules.
 - External clients do not need direct knowledge of internal container addresses.
- Network segmentation pattern:
 - `component_a_network` handles internal communication.
 - `gateway_network` isolates gateway-facing service exposure.
- Dedicated storage pattern:
 - Data services are isolated in dedicated containers per bounded domain.
 - This supports ownership and independent scaling/evolution decisions.

3. Layered Structure

Layered View



Layer definitions and relations

- Layer 1 - Presentation:
 - `frontend/web`, `frontend/cli`.
- Layer 2 - Entry and Communication:
 - `traefik` (gateway), `rabbitmq` (async transport).
- Layer 3 - Application and Domain Logic:
 - `user-service`, `music-service`, `achievements-service`, `notification-service`, `event-processor`.
- Layer 4 - Data:
 - PostgreSQL instances and CouchDB.

Dependency rule:

- Presentation depends on entry/communication layer.
- Domain services depend on data and asynchronous messaging layers.
- Direct cross-layer shortcuts are limited to defined API/message contracts.

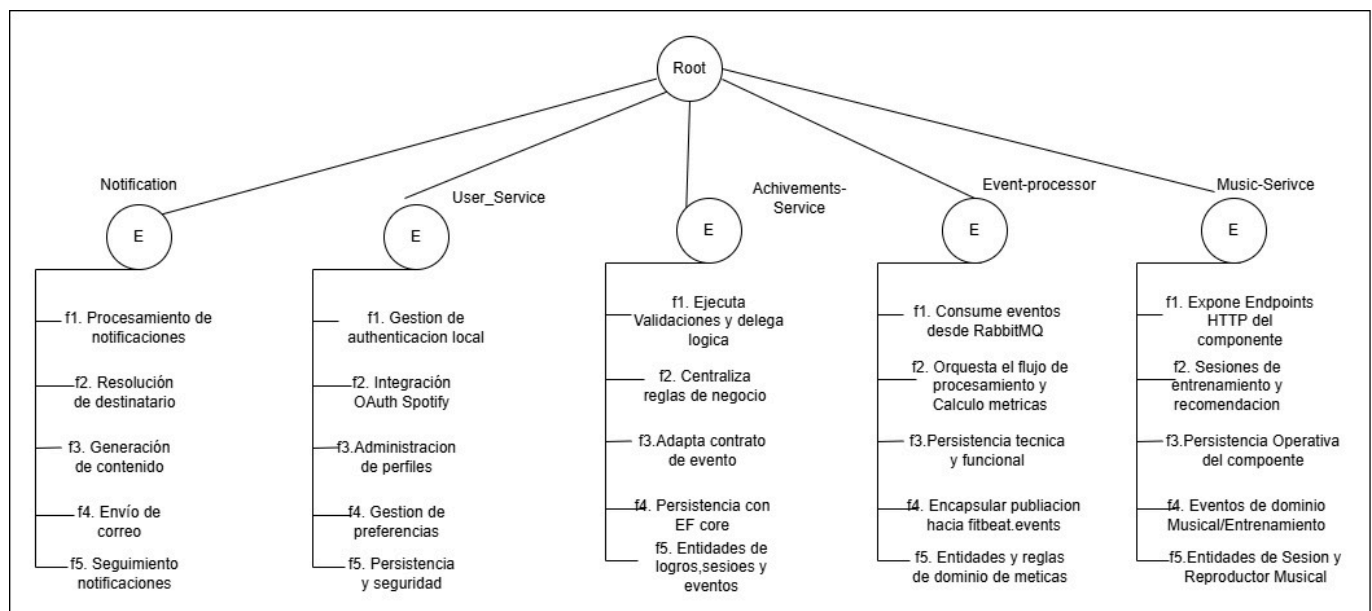
Layered patterns used

- N-tier (4-tier) organization:

- Enforces a high-level separation between UI, communication/orchestration, business logic, and data.
- Improves maintainability by reducing cross-cutting dependencies.
- API Gateway as communication boundary:
 - Concentrates external API access and routing concerns in one component.
 - Simplifies frontend integration and policy evolution.
- Broker-mediated asynchronous collaboration:
 - Event-based interactions are delegated to the messaging tier.
 - This reduces temporal coupling between producer and consumer services.
- Domain-oriented service layering:
 - Logic-tier services focus on distinct responsibilities and coordinate through contracts.
 - This preserves cohesion and limits impact radius of changes.

4. Decomposition Structure

Decomposition View



Decomposition of the system

- **FitBeat System**
 - **Presentation Subsystem**
 - Web Frontend (**frontend/web**)
 - CLI Frontend (**frontend/cli**)
 - **Identity Subsystem**
 - User Service (**user-service**)
 - **Workout and Playback Subsystem**
 - Music Service (**music-service**)
 - **Gamification Subsystem**
 - Achievements Service (**achievements-service**)
 - **Notification Subsystem**
 - Notification Service (**notification-service**)
 - **Async Processing Subsystem**

- Event Processor (**event-processor**)
- **Infrastructure Subsystem**
 - API Gateway (**traefik**)
 - Broker (**rabbitmq**)
 - Databases (PostgreSQL, CouchDB)
- **External Integration Subsystem**
 - Spotify API

Decomposition patterns used

- Functional decomposition by business capability:
 - The system is partitioned into identity, workout/playback, achievements, notifications, and async processing.
 - Each subsystem owns a bounded part of the platform behavior.
- Infrastructure decomposition:
 - Gateway, broker, and persistence are separated from domain services as dedicated infrastructure modules.
 - This clarifies operational responsibilities and runtime dependencies.
- Integration decomposition:
 - Spotify integration is treated as an external subsystem boundary.
 - External dependencies are isolated behind service interfaces.
- High cohesion and loose coupling principle:
 - Internal module responsibilities are focused.
 - Inter-service dependencies are mediated through REST/WebSocket/AMQP contracts.

Prototype

Local deployment instructions

1. Clone the repository:

```
git clone https://github.com/adrianyebid/FitBeat.git
cd FitBeat
```

2. Create **.env** from **.env.example** and fill required credentials:

- Spotify credentials (**SPOTIFY_CLIENT_ID**, **SPOTIFY_CLIENT_SECRET**).
- Database and messaging environment variables.

3. Build and run all services:

```
docker-compose up --build
```

4. Check container status:

```
docker-compose ps
```

5. Main access points:

- Web frontend: <http://localhost:5173>
- API Gateway: <http://localhost:8090>
- Traefik dashboard: <http://localhost:8088>
- RabbitMQ dashboard: <http://localhost:15672>

High-level functional flow

1. User signs up or logs in.
2. User connects Spotify account.
3. User configures workout/music preferences.
4. User starts training session.
5. Playback is controlled in real time.
6. Domain events are emitted and consumed asynchronously.
7. User checks achievements and notifications.

Notes

- Export this file to PDF as [p2_1e.pdf](#).
- Ensure all required images are embedded in the final exported document:
 - Logo
 - C&C View
 - Deployment View
 - Layered View
 - Decomposition View